

Übungsblatt 6

Das SOS-Problem ist wie folgt definiert:

- Problem:** Sum-of-Subset (SOS)
Eingabe: Natürliche Zahlen $a_1, \dots, a_n$ sowie eine weitere natürliche Zahl b
Frage: Gibt es eine Menge $I \subseteq \{1, \dots, n\}$, so dass $\sum_{i \in I} a_i = b$?
Mit anderen Worten: Gibt es Zahlen aus $\{a_1, \dots, a_n\}$, die sich zu b aufsummieren (wobei jedes a_i höchstens einmal als Summand auftreten darf)?

1. (a) Schreiben Sie in C++ eine Funktion mit dem Kopf

```
bool isGoodSolutionForSOS(int n, int* a, int b, int solutionSize, int* solution)
```

die entscheidet, ob ein gegebener Lösungskandidat einer Instanz des SOS-Problems die Instanz tatsächlich löst. Die SOS-Instanz ist dabei durch die Parameter n , a und b gegeben, wobei n angibt, wie viele Elemente sich in dem durch a gegebenen Array befinden.

Die Potentielle Lösung ist durch $solutionSize$ und $solution$ gegeben, wobei $solution$ Indizes von Elementen von a enthält. Die Frage, die die Funktion beantworten soll ist, ob die Summe der Elemente aus a , deren Indizes in $solution$ stehen, gerade gleich b ist.

- (b) Schätzen Sie die Laufzeit Ihrer Funktion in Abhängigkeit von n nach oben ab! Geben Sie also eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ an, so dass die Anzahl der Elementaroperationen, die ausgeführt werden, wenn Ihre Funktion mit maximal n Elementen in a aufgerufen wird, nie $f(n)$ überschreitet. Bleiben Sie konservativ bei Ihrer Abschätzung, machen Sie also f nicht unnötig groß!

Zu den Elementaroperationen bitte den Text am Ende der Aufgabenliste beachten!

2. (a) Schreiben Sie in C++ eine Funktion mit dem Kopf

```
bool SOS(int n, int* a, int b)
```

Die Funktion soll genau dann `true` zurückgeben, wenn sich Elemente aus a finden, die sich gemäß der Definition von SOS zu b aufsummieren lassen und `false` sonst.

Im Moodle findet sich eine Datei `SOSTests.cpp`, die eine Reihe Funktionen enthält, die Sie zum Testen Ihrer Lösung benutzen können.

- (b) Bestimmen Sie, analog zur 1. Aufgabe, die Laufzeit der von Ihnen programmierten Funktion in Abhängigkeit von n . Sie dürfen Ihre Abschätzung in O -Notation angeben.

Als Elementaroperationen zählen:

- Wertzuweisungen
- Arithmetische oder logische Operationen
- Vergleichsoperationen
- Das `return`-Statement (da dort implizit eine Wertzuweisung versteckt ist)
- Funktionsaufrufe:

Jedes in einem Funktionsaufruf übergebene Element impliziert eine Wertzuweisung und zählt daher ebenfalls als Elementaroperation. Diese Elementaroperationen werden am Ort des Aufrufs der Funktion gezählt, nicht in der aufgerufenen Funktion. Für die Abschätzung der Laufzeit der Funktion `isGoodSolutionForSOS` muss also z.B. der „Kopf“ der Funktion nicht mitgezählt werden. Wird aber z.B. diese Funktion in `SOS` (Aufgabe 2) aufgerufen (was nicht zwangsläufig sein muss!), so gehen alle Elementaroperationen, die sich aus dem Funktionsaufruf ergeben, in die Berechnung mit ein. (nächste Seite beachten :))

- Der Einfachheit wegen verzichten wir darauf Operationen zur Speicherverwaltung (etwa Variablendeklarationen oder Anlegen eines neuen Arrays mit `new`) mitzuzählen. Derlei Operationen benötigen zwar auch Zeit, sie mitzuzählen verkompliziert die Aufgabe aber unnötig.